COMSATS UNIVERSITY ISLAMABAD
Department of Computer Science

CYC386 — Secure Software Design and Development
MIDTERM LAB EXAM

1-Week DevSecOps Security Engineering Sprint

Fall 2026 Semester

Instructor: Engr. Muhammad Ahmad Nawaz | Office: 1st Floor CS Faculty Sitting Area | Email: m.ahmadnawazz@gmail.com

Team Size	Maximum 3 members	**Time Allowed**	7 Calendar Days (industry-style time-boxed sprint)
Submission	GitHub + Jira Board + MS Teams	**Assessment**	100 Marks (+15 Bonus)
Primary CLOs	CLO-5 (Applying) & CLO-6 (Creating)	**GAs**	GA 2, 3, 4, 6

These guidelines are fully aligned with the official Course Description Form (CDF) of CYC386 and the lab manual. The exam is designed as a real-world DevSecOps Security Engineering Sprint to develop attack-resistant software in a team environment. Because the sprint lasts one full week and teams are capped at three members, the technical scope is intentionally large, multi-module, and industry-justifiable.

1. COURSE LEARNING OUTCOMES (CLOs) & GRADUATE ATTRIBUTES (GAs)

CLO	Description	Bloom's Level	GA
CLO-5 (Lab)	Analyze and test software to identify vulnerabilities, mitigate threats, and ensure secure solutions.	Applying	GA 2, 3
CLO-6 (Lab)	Develop an attack-resistant software in a team environment.	Creating	GA 2–4, 6

The sprint also supports theory CLOs 1–4 (Secure SDLC, Requirement Analysis, Secure Design Considerations, Security Frameworks & Models, Code Vulnerabilities, Testing, AppSec frontiers). Every deliverable must explicitly reference the **CIA triad** (Confidentiality, Integrity, Availability).

2. REAL-WORLD OBJECTIVE (INDUSTRY-ORIENTED SPRINT)

You are a **DevSecOps Security Engineering Team** embedded in a fast-paced product company. A client (or internal product owner) has handed you an existing or starter application that must be hardened to production-grade security within **seven calendar days**. A formal security review and demo will be conducted at the end of the sprint.

Mission Statement (7-Day Sprint)

- Perform rigorous **Protection Needs Elicitation (PNE)** and map every protection need to the CIA triad (CDF Unit 2).
- Execute complete **Threat Modeling** using STRIDE, Data-Flow Diagrams (context + level-1 minimum), Attack Trees, and **CVSS v3.1** quantitative risk scoring (CDF Unit 3).
- Identify, safely demonstrate (lab-only), and fully remediate a minimum set of vulnerability classes: **Buffer Overflow**, **Command Injection**, **IDOR**, **CSRF**, and **Clickjacking**, plus any additional high-impact findings discovered during testing (CDF Unit 5).
- Implement defensive coding practices, automated security gates (SAST + DAST), and a production-ready **CI/CD pipeline** using GitHub Actions that fails the build on critical findings.
- Manage the entire effort with professional **Git / GitHub** workflow (branching, protected main, PRs, code reviews) and a live **Jira** (or equivalent) sprint board showing full traceability from PNE → Threat → Fix → Test.
- Deliver a deployable, attack-resistant application together with professional documentation and a technical demo that would satisfy a real security review.

3. PROJECT THEMES & CUSTOM PROPOSALS

Teams may choose one of the recommended themes below **or propose their own theme**. Custom themes are strongly encouraged provided they satisfy all mandatory technical requirements (see Section 6). Theme selection / custom proposals must be submitted within the first 24 hours and will be approved on a first-come, first-served uniqueness basis. No two teams may work on identical or near-identical themes.

3.1 Recommended Themes (Starter Guidance)

Domain	Suggested Base / Focus	Technical Emphasis Required
Web Application / CMS	Flask, Django, or Node blog/CMS with file handling	Command Injection (file/system ops), Buffer Overflow in native helper or image/parser extension, IDOR, CSRF, Clickjacking
REST / GraphQL API Platform	FastAPI, Flask, Express, or NestJS JWT-protected API	IDOR on resources, Command Injection via system/subprocess, Buffer Overflow in native library binding, token/CSRF issues
File Sharing / Collaboration	Upload + share service (Flask/Node)	Command Injection on filenames/paths, Buffer Overflow in parsers, IDOR on shared objects, CSRF on share actions

Password / Secrets Vault	Custom or Passky-like vault service	Buffer Overflow in crypto/memory helpers, Command Injection, IDOR on vault entries, CSRF on critical actions
IoT / Edge Device Dashboard	Dashboard + simulated C/C++ or Go device agent	Buffer Overflow in agent, Command Injection in device command path, IDOR on device IDs, Clickjacking on console
Secure Messaging / Chat	WebSocket/Socket.IO chat + media handling	Buffer Overflow in media/parser component, Command Injection, IDOR on rooms/messages, CSRF
CI/CD or DevOps Tooling Portal	Internal dashboard that triggers builds/scripts	Command Injection (pipeline scripts), Buffer Overflow in helpers, IDOR on job/artifact access, CSRF on trigger actions
E-Learning / LMS Module	Course/assignment submission platform	Command Injection on file processing, IDOR on submissions/grades, CSRF, Clickjacking, optional native grading helper (Buffer Overflow)

3.2 Custom Theme Rules (Encouraged)

• Submit a ½-page proposal within the first 24 hours describing the application domain, planned architecture, and how each mandatory vulnerability class will be introduced and fixed.
• The proposal must show that Buffer Overflow can be demonstrated realistically (native code, C/C++ extension, unsafe parser, or controlled memory-unsafe component).
• Command Injection must have a realistic attack surface (system calls, shell, external process invocation, or equivalent).
• IDOR, CSRF, and Clickjacking must be applicable to the chosen interface (web UI and/or API).
• Instructor approval is required; uniqueness across the class is enforced.
• Once approved, the custom theme carries the same weight and rubric as recommended themes.

4. 7-DAY SPRINT TIMELINE & DETAILED PHASES

Day	Phase	Mandatory Activities & Outputs
Day 0–1	Phase 1: PNE, CIA Mapping & Project Setup	Asset inventory; Protection Needs Elicitation; explicit CIA mapping for every need; Jira project + sprint board creation; GitHub repo initialization (protected main, branch protection rules); theme approval / custom proposal; high-level architecture sketch
Day 1–2	Phase 2: Threat Modeling & Quantitative Risk	Context + Level-1 DFDs; complete STRIDE analysis per element; Attack Tree(s) for at least two critical threats; CVSS v3.1 scoring for all high/critical items; prioritized risk register linked to Jira issues; residual risk discussion
Day 2–5	Phase 3: Secure Implementation (Core Engineering)	Implement and prove fixes for Buffer Overflow, Command Injection, IDOR, CSRF, Clickjacking; additional findings as discovered; feature branches + Pull Requests with mandatory code review; continuous meaningful commits; secure coding checklist compliance
Day 5–6	Phase 4: CI/CD Pipeline & Automated Security Testing	GitHub Actions workflow (build, unit tests, SAST, DAST); fail-on-critical policy demonstrated; CodeQL / SonarCloud / Semgrep + OWASP ZAP (or equivalent) evidence; before/after comparison; pipeline logs retained in repo
Day 6–7	Phase 5: Documentation, Demo, Hardening & Submission	Final IEEE-style report; 6–10 min technical demo video; Jira sprint closure evidence; repository hygiene (README, SECURITY.md, license if applicable); final security self-assessment against CIA and residual risks

5. MANDATORY DELIVERABLES

All artifacts below must reside in (or be clearly linked from) the team GitHub repository. Jira board access or exported evidence is required.

• **GitHub Repository** named `SecureApp-Sprint-<TeamName>`: protected main branch, feature-branch workflow, Pull Requests with at least one peer review per PR, meaningful commit messages, and a clear README describing how to build, run, and test the secured application.
• **Jira (or approved equivalent) Board**: Sprint board with Epics → Stories → Tasks. Every PNE item, threat, and fix must be traceable. Provide view access or PDF/screenshot export of the completed board.
• **PNE_Report.md** (or PDF): Asset inventory, elicited protection needs, stakeholders, and explicit mapping of each need to Confidentiality / Integrity / Availability (CDF Unit 2).
• **THREAT_MODEL.pdf**: Minimum Context + Level-1 DFDs, complete STRIDE table, at least one Attack Tree, full CVSS v3.1 risk assessment table, and residual risk statement (CDF Unit 3).
• **SECURITY_IMPLEMENTATION.md**: For each of Buffer Overflow, Command Injection, IDOR, CSRF, and Clickjacking — vulnerable code snippet or description, exploit/proof-of-concept evidence (lab-only), secure fix with code, and verification steps. Screenshots required.
• **SECURITY_TEST_REPORT.md**: SAST results (tool, configuration, findings, remediation status) + DAST results (OWASP ZAP or equivalent) with before/after evidence. Pipeline must demonstrably fail on critical severity.
• **.github/workflows/ci-cd.yml** (and supporting workflow files): Working pipeline triggered on push and pull_request. Must include build, test, and at least one security scanning stage.
• **Final Report (PDF, 18–30 pages, IEEE-style)**: Executive summary, architecture, PNE summary, threat model summary, detailed remediation, testing evidence, CIA residual risk analysis, lessons learned, and individual contribution statement.
• **Demo Video (6–10 minutes)**: Screen recording with narration covering architecture, live demonstration of at least three vulnerability classes (before → after), pipeline, and key design decisions.

6. DETAILED TECHNICAL & INDUSTRY REQUIREMENTS

6.1 Process, Git, GitHub & Jira

• Branching strategy: main (protected), develop (optional), feature/* and fix/* branches. Direct pushes to main must be disabled.
• Every feature or fix lands via Pull Request. At least one team member must review and approve each PR (evidence in GitHub).
• Commit messages must be descriptive. Squash or rebase is acceptable; history must remain understandable.
• Jira (or GitHub Projects / Linear with instructor approval): Issues must link to PNE items and to STRIDE threats. Sprint board must show Done / In Progress / To Do states at submission time.
• SECURITY.md and a basic CONTRIBUTING.md are expected in the repository root.

6.2 Protection Needs Elicitation & CIA Triad

• Identify assets (data, processes, components, external entities).
• Elicit protection needs from stakeholder perspectives (user, admin, operator, attacker).
• Map every protection need to one or more of Confidentiality, Integrity, Availability. This mapping must appear in both the PNE report and the final report.
• Derive measurable security requirements (e.g., “object-level authorization on every resource access”, “no shell interpretation of user-controlled strings”).

6.3 Threat Modeling & Risk Assessment

• Produce at least a Context Diagram and a Level-1 Data-Flow Diagram.
• Apply STRIDE systematically to each DFD element (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege).
• Construct Attack Tree(s) for a minimum of two high-impact threats (e.g., remote code execution via Command Injection or privilege escalation via Buffer Overflow).
• Score all High and Critical threats with CVSS v3.1 (Base Score + optional Temporal/Environmental). Justify vector strings.
• Maintain a prioritized risk register and link each entry to a Jira issue and to a remediation commit/PR.

6.4 Mandatory Vulnerability Classes — Technical Expectations

The following five classes are mandatory. Teams must both demonstrate the vulnerability (controlled lab environment only) and prove the fix.

Vulnerability	Minimum Technical Expectation
Buffer Overflow	Present a realistic memory-unsafe component (C/C++ function, native extension, unsafe parser, or equivalent). Show overflow of a buffer that can influence control flow or data. Fix with bounds checking, safe APIs (strncpy_s / snprintf / span, etc.), and demonstrate compiler mitigations (stack canary, ASAN, or equivalent). Provide before/after evidence.
Command Injection	Identify a code path that invokes a shell or external process with user-controlled input. Demonstrate injection (e.g., ; | && ` $()). Remediate by eliminating shell interpretation (execve-style argument arrays, parameterized APIs, allow-lists). Prove the injection no longer succeeds.
IDOR (Insecure Direct Object Reference)	Show access to another user’s or unauthorized object by manipulating an identifier. Implement object-level authorization checks on every sensitive access. Prove that unauthorized IDs are rejected.
CSRF	Demonstrate a state-changing request that succeeds without a valid anti-CSRF token (or equivalent). Implement synchronizer tokens (or double-submit cookie) + SameSite cookie attributes and verify rejection of forged requests.
Clickjacking	Show that a sensitive page can be framed. Apply X-Frame-Options (DENY or SAMEORIGIN) and/or CSP frame-ancestors 'none' (or equivalent). Verify framing is blocked.

Additional vulnerabilities discovered during SAST/DAST (XSS, SSRF, insecure deserialization, etc.) should be fixed and documented for higher scores and bonus consideration.

6.5 Secure Coding & Compiler / Runtime Defenses

• Prefer memory-safe languages or isolated unsafe modules with clear boundaries.
• Enable and document compiler/runtime protections where applicable: -fstack-protector-strong, -D_FORTIFY_SOURCE=2, ASLR, NX/DEP, ASAN during testing.
• Never trust client-supplied length or size fields for memory operations.
• Centralize authorization and input validation; avoid scattered ad-hoc checks.

6.6 CI/CD & Automated Security Testing

• GitHub Actions workflow must run on push and pull_request to main/develop.
• Stages (minimum): checkout → build → unit/integration tests → SAST → (optional) DAST or container scan.
• Pipeline must exit non-zero (fail the check) when a critical or high severity finding is present in the security stage.
• Recommended tools: CodeQL, SonarCloud, Semgrep, Bandit, ESLint security plugins, OWASP ZAP baseline/full scan, Trivy/Grype for containers if used.
• Retain workflow run logs and SARIF/JSON reports as artifacts or committed evidence.

7. EVALUATION RUBRIC (INDUSTRY-FOCUSED)

Area	Assessment Focus	Marks
PNE + CIA Mapping	Completeness of asset inventory, quality of elicited needs, explicit and consistent CIA linkage, Jira traceability	12
Threat Modeling & Risk Assessment	DFD quality, systematic STRIDE, Attack Tree depth, correct CVSS vectors, prioritized register linked to work items	15

Secure Implementation (5+ vuln classes)	Correctness and evidence of Buffer Overflow, Command Injection, IDOR, CSRF, Clickjacking fixes; additional high-value fixes; code quality	28
CI/CD Pipeline & Automation	Working GitHub Actions, fail-on-critical behavior, SAST/DAST integration, Git branching & PR discipline	15
Security Testing Evidence	Meaningful SAST/DAST before/after results, remediation proof, pipeline logs	12
Documentation & Professionalism	IEEE-style report structure and depth, Jira board completeness, repository hygiene (README, SECURITY.md)	10
Demo & Technical Viva	Clarity of demo, ability to justify design decisions, residual risk awareness, individual contribution evidence	8

Bonus (Advanced DevSecOps) up to +15: Dockerized / containerized deployment with security scanning, high-maturity pipeline (caching, matrix builds, multiple security gates), formal verification or model-checking notes, exceptional Jira/Git hygiene and automation, or innovative mitigations beyond the mandatory set.

Partial credit is awarded for correct analysis and partial fixes accompanied by clear evidence. Code that does not build or tests that cannot be reproduced receive substantially reduced marks.

8. SUBMISSION CHECKLIST & DEADLINE

- Primary: GitHub repository (invite instructor as collaborator if private) containing all artifacts listed in Section 5.
- Secondary: Upload Final Report PDF and Demo Video (or private link) via the designated MS Teams assignment.
- Jira: Grant view access or attach exported board PDF/screenshots.
- Deadline: End of Day 7 (exact date and time announced on MS Teams). Late penalty: –20 % per calendar day (maximum two days).
- Each member must have visible, meaningful commits and PR participation. Free-riding will be identified in viva and penalized individually.

9. ACADEMIC INTEGRITY & ETHICAL BOUNDARIES

All vulnerability demonstration and exploitation activities are restricted to systems and applications under the team's direct control for this exam. Attacking university infrastructure, third-party services, or any system without explicit written authorization is strictly forbidden and will be reported under university academic-integrity and cybersecurity policies. The techniques practiced are solely for defensive education and the construction of attack-resistant software. Plagiarism of code, reports, or threat models from other teams or external sources without proper citation will result in zero marks for the affected component and may trigger further disciplinary action. Use of AI coding assistants is permitted provided the team can fully explain and justify every security-critical change during viva.

10. SUGGESTED REFERENCES & RESOURCES

- Official CYC386 Course Description Form (CDF) and associated Lab Manual.
- OWASP Top 10, OWASP Testing Guide, OWASP Cheat Sheet Series (Buffer Overflow, OS Command Injection, CSRF, Clickjacking, Authorization).
- CWE-119 / CWE-120 (Buffer Errors), CWE-78 (OS Command Injection), CWE-639 (Authorization Bypass / IDOR), CWE-352 (CSRF), CWE-1021 (Improper Restriction of Rendered UI Layers / Clickjacking).
- Microsoft / OWASP STRIDE resources; Attack Tree literature; CVSS v3.1 Specification.
- GitHub Actions documentation; CodeQL, SonarCloud, Semgrep, OWASP ZAP, Trivy documentation.
- Lecture materials on Memory Corruption, Input-Based Vulnerabilities, Secure Coding, and DevSecOps practices by Engr. M. Ahmad Nawaz.
- Secure coding references: CERT C/C++ Coding Standard, "Building Secure Software", "Designing Secure Software" (course text list).

— End of Midterm Lab Exam Guidelines (Fall 2026) —

"Deliver software that would pass a real security review. Secure coding is a mindset, not just a toolset." — Engr. M. Ahmad Nawaz